Final Report – Hardware Security Assessment

Client: Thanos Tech – Node Device Hardware Security Evaluation

Kevin Umba

COSC 6840 Ethical Hacking Theory/Practice

12/09/2025

Contents

Introduction

Client Overview

Thanos Tech contracted Kevin Umba to perform a hardware security evaluation on its newest IOT device, the Node. The goal of this engagement was to assess the Node's resilience against physical attacks.

Scope of Work

The devices to analyze for this engagement include the following:

1. STM32F103C8T6 Microcontroller (MCU)
2. Winbond W25Q64JV External SPI Flash
3. Subcomponents to exploit:
    a. UART debug shell
    b. Internal & External flash Memory
    c. Undocumented UART data stream

The engagement terms included the following domains:

- Hardware reconnaissance
- UART enumeration
- Password, internal and external memory extraction
- Reading undocumented data streams

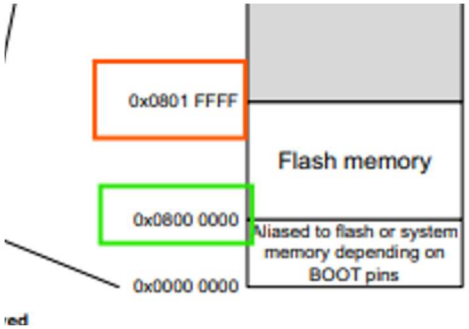
Objectives

1. Identify and connect to the engineering debug shell.
2. Extract and crack existing credentials.
3. Assess memory encryption
4. Extract internal SWD and external SPI flash memory.
5. Identify and read hidden data stream to other physically connected devices.

Methodology

Reconnaissance

- The Node Github Repository
- Manufacturer Datasheets:
    - STM32F103C8T6 microcontroller:
        - Outlined the memory mapping to the internal flash memory.



        - UART pin hiding the following data streams:

- Debug Shell: USART2 default pins (PA2-TX, PA3- RX)

											ETR[1]	
H2	M2	11	H2	15	24	8	PA1	I/O	-	PA1	USART2_RTS[9] ADC12_IN1/ TIM2_CH2[9]	-
J2	K3	12	F3	16	25	9	PA2	I/O	-	PA2	USART2_TX[9] ADC12_IN2/ TIM2_CH3[9]	-
K2	L3	13	G3	17	26	10	PA3	I/O	-	PA3	USART2_RX[9] ADC12_IN3/ TIM2_CH4[9]	-
E4	E3	-	C2	18	27	-	V_{SS_4}	S	-	V_{SS_4}	-	-
F4	H3	-	D2	19	28	-	V_{DD_4}	S	-	V_{DD_4}	-	-

- Hidden Data Stream: USART1 default pins (PB10 TX, PB11 RX)

-	PE14	I/O	FT	PE14	-	TIM1_CH4
-	PE15	I/O	FT	PE15	-	TIM1_BKIN
-	PB10	I/O	FT	PB10	I2C2_SCL/ USART3_TX[9]	TIM2_CH3
-	PB11	I/O	FT	PB11	I2C2_SDA/ USART3_RX[9]	TIM2_CH4
18	V_{SS_1}	S	-	V_{SS_1}	-	-
19	V_{DD_1}	S	-	V_{DD_1}	-	-

  - Winbond W25Q64JV external flash chip:
    - Used to understand the pin configuration to access the external flash memory CH341A.

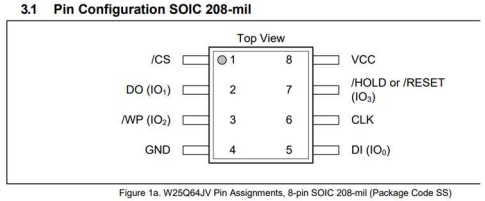


Figure 1a. W25Q64JV Pin Assignments, 8-pin SOIC 208-mil (Package Code SS)

Finding 1 – Exposed UART Debug Shell

Description

The STM microcontroller exposed the debug shell through unprotected UART port on the default pins (PA2, PA3) at 9600 baud rate.

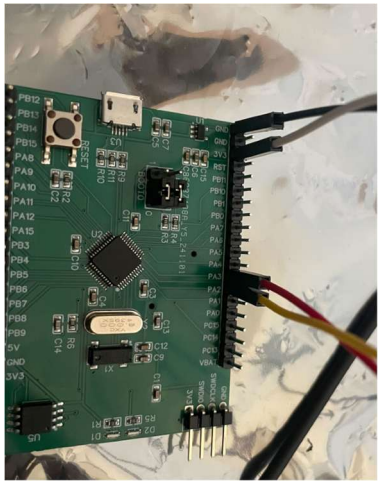
Tools

- Host: Ubuntu 22.04 LTS (WSL)
- FT232 USB-UART adapter
- Commands:

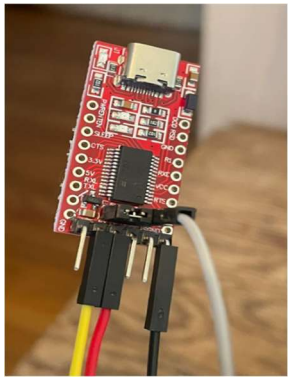
    - screen – terminal multiplexer for serial communication
    - usbipd – bind physical USB to WSL
- Baud rate 9600bps

Method

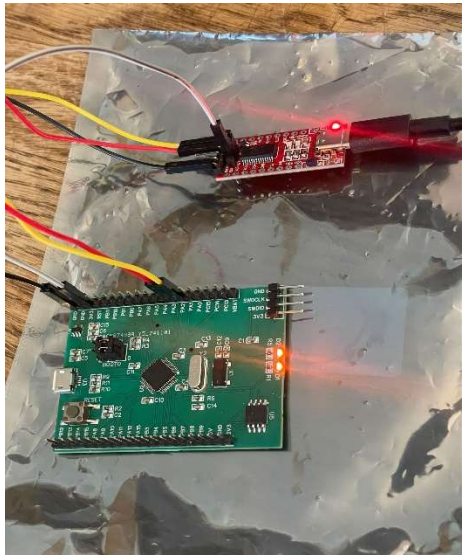
1. **Hardware Setup**
    - Locate the Ground, 3.3v pins on the microcontroller and the UART adapter to supply power.



    - Locate the RX and TX pins on the UART adapter



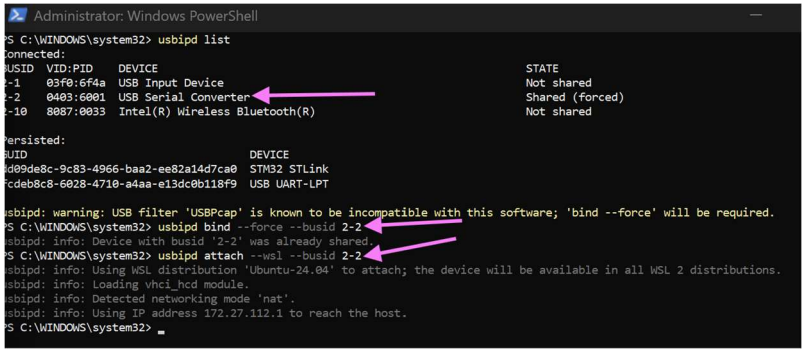
- Connect the pins with jumper wires



2. **Debug Shell access**

- Binding the USB to WSL:
    - List connect USB ports
      Command: usbipd list
    - Bind and connect the detected device to WSL
      Command:
      usbipd bind --busid <corresponding id>

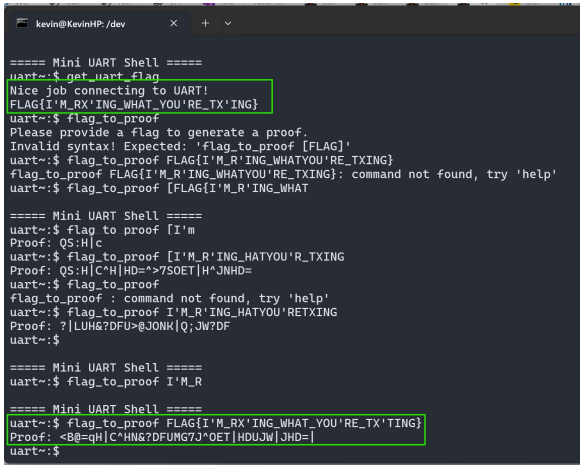
usbipd attach --wsl --build <corresponding id>



- Connect to the UART adapter
  Command: sudo screen /dev/ttyUSB 9600

3. Flag 1 and Proof

- Execute the following command to get the first flag
  Command: get_uart_flag
  Flag{I'M_RX'ING_WHAT_YOU'RE_TX'ING}
  Proof: HKOIzW)LmTW5KMUaVVCSm[Nc)QSaSf)H\I|



Finding 2 - Password Extraction

Description

The STM microcontroller exposed the debug shell through unprotected UART port on the default pins (PA2, PA3) at 9600 baud rates. A password hash was stored in the debug shell. The password was not protected by authentication; the hash was easily broken with the rockyou list.

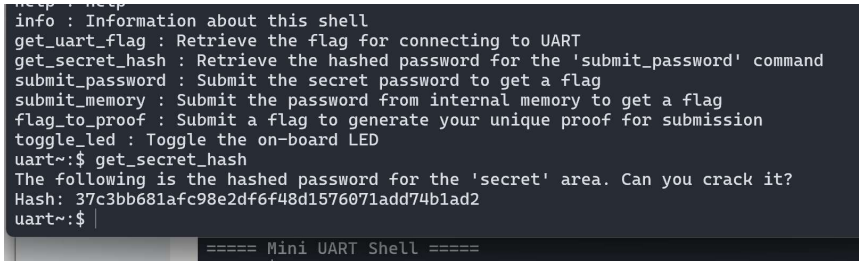
Tools

- Commands: John the Ripper– crack the password hash

Method

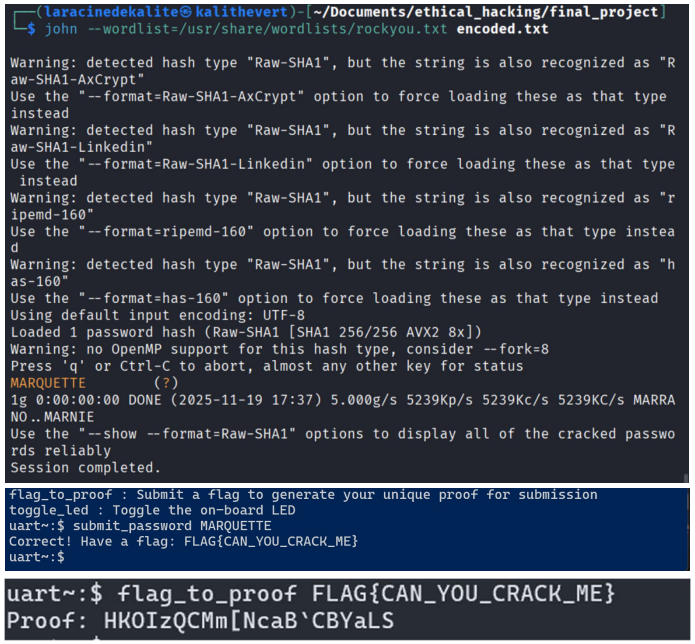
1. **Accessing Password Hash**

- Execute the following command to reveal the password hash:
  Command: get_secret_hash
  Password hash: 37c3bb681afc98e2df6f48d1576071add74b1ad2



2. **Password Cracking**

- Use the rock you list to crack the password hash.
  Command: john –wordlist=/usr/share/wordlist/rockyou.txt <password path>
  Original Password: MARQUETTE
  Flag{CAN_YOU_CRACK_ME}
  Proof: HKOIzQCMm[NcaB`CBYaLS



Finding 3– Internal Flash Memory

Description

The STM microcontroller was shipped with SWD enabled without readout protection, allowing memory flash dump. The address collected in the manufacturer datasheet facilitated the extraction.

Tools

- ST-Link V2 – debugger to enable SWD connection
- Commands:
  - openOCD: Open On-Chip Debugger for SWD interface
  - telnet: network protocol for OpenOCD server connection
  - HexedIT – inspect human readable characters to extract the flag

- Memory Addresses:
    - Start address: 0x08000000
    - End address: 0x0801FFFF
    - Memory Byte size: 131,072bytes
      End address – start address + 1= bytes
      0x0801FFFF – 0x0800000 + 1 = 0x02000 which is the same as 131,072bytes

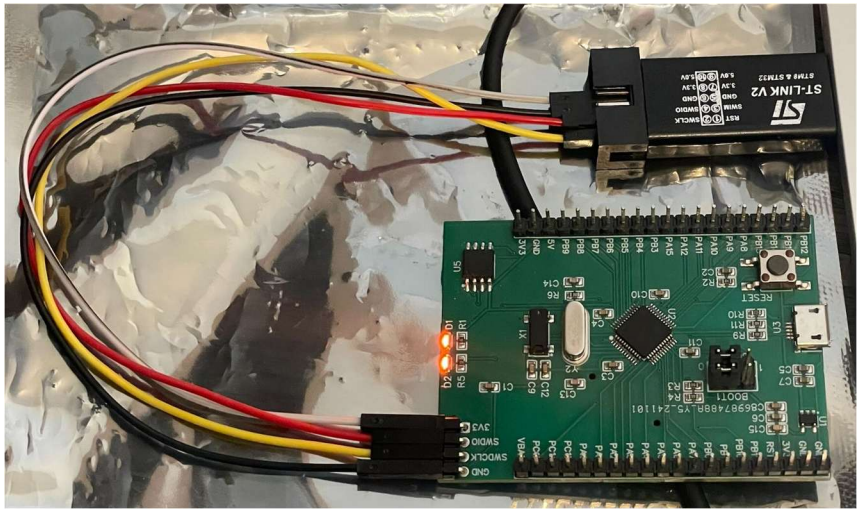
Method

1. **Hardware Setup**

- Locate the SWCLK, and SWDIO pins for communication and the ground and 3.3V pins for power on the ST-Link V2

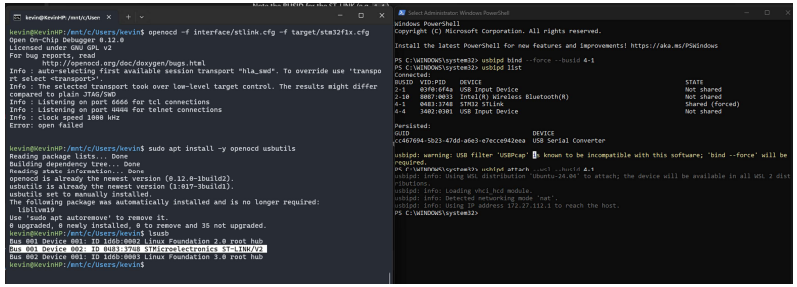
- Locate the corresponding SWCLK, SWDIO, GND, and 3.3V pins on the STM microcontroller

- Connect the pins with jumper wires

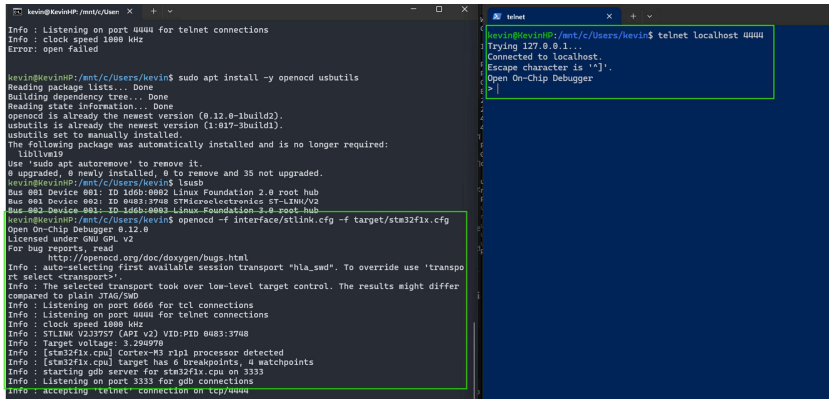


2. Establish the debug connection

- List connected USB devices on WSL
  Command: lusb

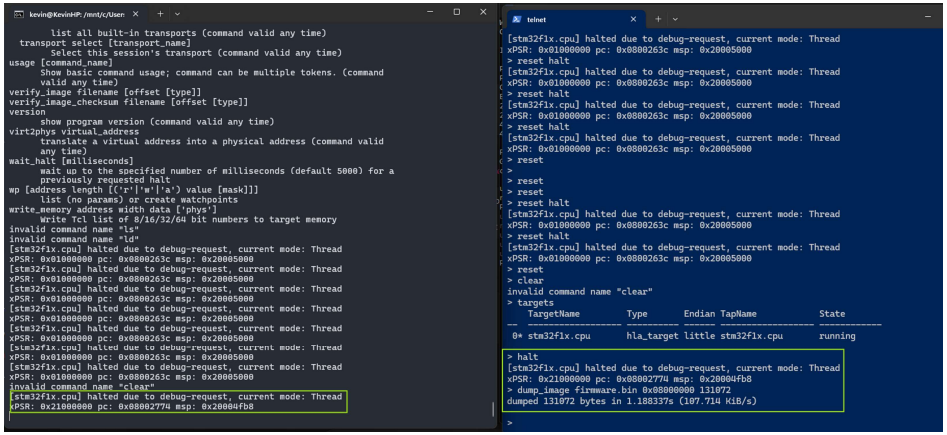


- Connect to the debugger via openocd and telnet on port 4444
  Command:
  openocd -f interface/stlink.cfg -f target/stm32f1x.cfg
  telnet localhost 4444



3. Memory dump

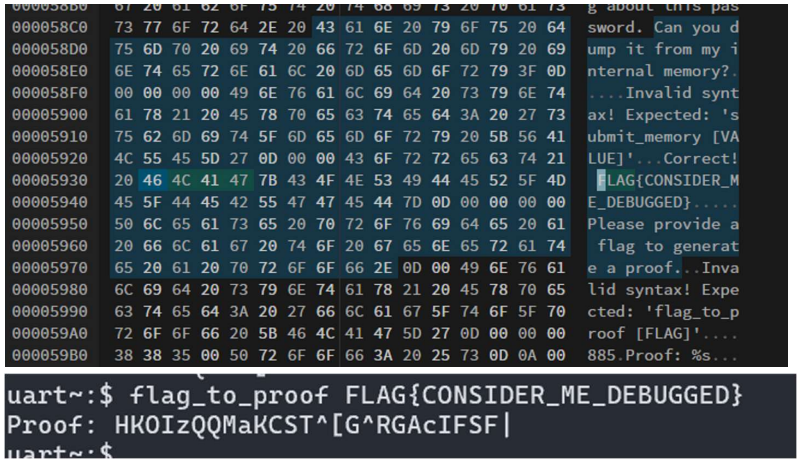
- Pause the pcb to read the internal memory
  Command: halt
- Use the memory addresses and byte size to dump the memory
  Command: dump_image firmware.bin 0x08000000 131072



4. Flag 3 and Proof

- Upload the memory dump to hexed it and search for human readable text: Link: hexed.it
  Flag{CONSIDER_ME_DEBUGGED}

Proof: HKOIzQQMaKCST^[G^RGAcIFSF|



Finding 4 – External SPI Flash Memory

Description

The Winbond flash storage was unencrypted and easily accessible using a CH341A programmer.

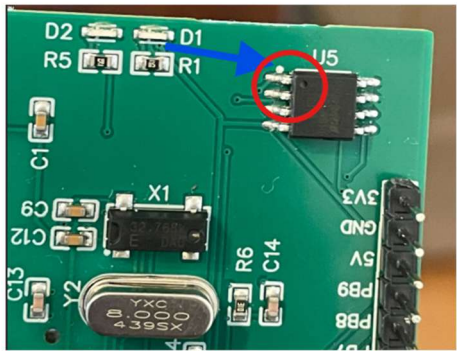
Tools

- CH341A– flashing and debugging programmer for SPI communication with external flash memory
- Commands:
  - flashrom: utility for flash chip operations
  - HexedIT – inspect human readable characters to extract the flag

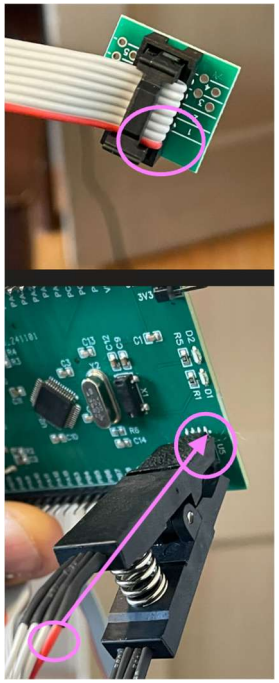
Method

1. **Hardware Setup**

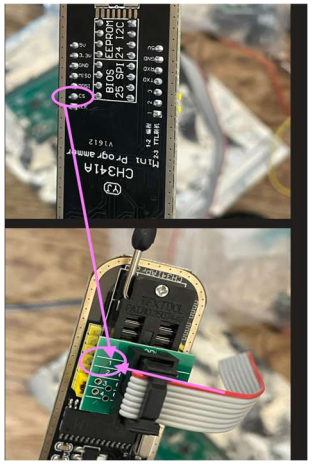
- Locate the CS pin on the Winbond chip marked by a white dot



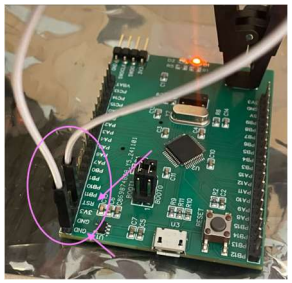
- Align the CH341A clip to the CS pin for proper connectivity. The red wire should match the white dot identified on the U5 chip above.



- Connect the clip to the CS pin on the programmer



- To read the external memory, the PCB execution must be paused. Connect the GND and RST pins to pause the execution.



2. Connect to the CH341A

- Identify the device in wsl
  Command: lusb

```
kevin@KevinHP:~/marquette/ethical_hacking/final$ lsusb
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 001 Device 003: ID 1a86:5512 QinHeng Electronics CH341 in EPP/MEM/I2C mode, EPP/I2C adap
ter
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
kevin@KevinHP:~/marquette/ethical_hacking/final$
```

- Read the external flash memory with flashrom
  Command: flashrom -p ch341a_spi -c “W25A64JV-.Q” -r dump.bin

```
kevin@KevinHP:~/marquette/ethical_hacking/final$ flashrom -p ch341a_spi -c "W25Q64JV-.Q" -r dump.bin
flashrom unknown on Linux 6.6.87.2-microsoft-standard-WSL2 (x86_64)
flashrom is free software, get the source code at https://flashrom.org

Using clock_gettime for delay loops (clk_id: 1, resolution: 1ns).
Found Winbond flash chip "W25Q64JV-.Q" (8192 kB, SPI) on ch341a_spi.
===
This flash part has status UNTESTED for operations: WP
The test status of this chip may have been updated in the latest development
version of flashrom. If you are running the latest development version,
please email a report to flashrom@flashrom.org if any of the above operations
work correctly for you with this flash chip. Please include the flashrom log
file for all operations you tested (see the man page for details), and mention
which mainboard or programmer you tested in the subject line.
Thanks for your help!
Reading flash... done.
```

3. **Flag 4 and Proof**

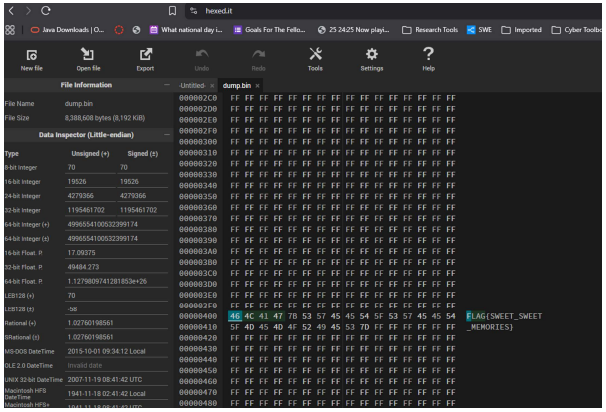
- View the content of the binary dump in hexedit

  Link: Hexed.it

  FLAG{SWEET_SWEET_MEMORIES}

  Proof: HKOIzaYDSV^aYDSV^[GL]THSU|

```
uart~:$ flag_to_proof FLAG{SWEET_SWEET_MEMORIES}
Proof: HKOIzaYDSV^aYDSV^[GL]THSU|
```



Finding 5 – Hidden Data Stream

Description

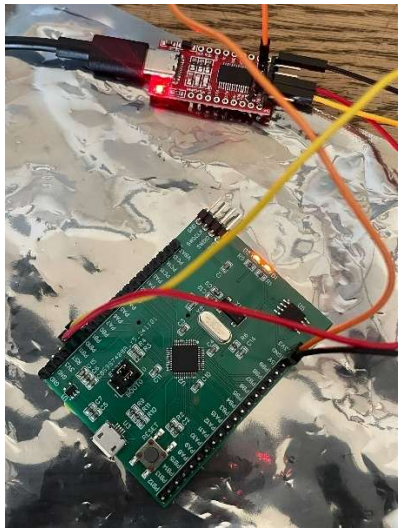
The undocumented UART data stream was detected on USART3 on default pins PB10, PB11. The seemingly unused UART peripherals were likely to contain additional communication.

Tools

- FT232 USB-UART adapter
- Commands:
    - screen – terminal multiplexer for serial communication
- Baud rate 115200bps

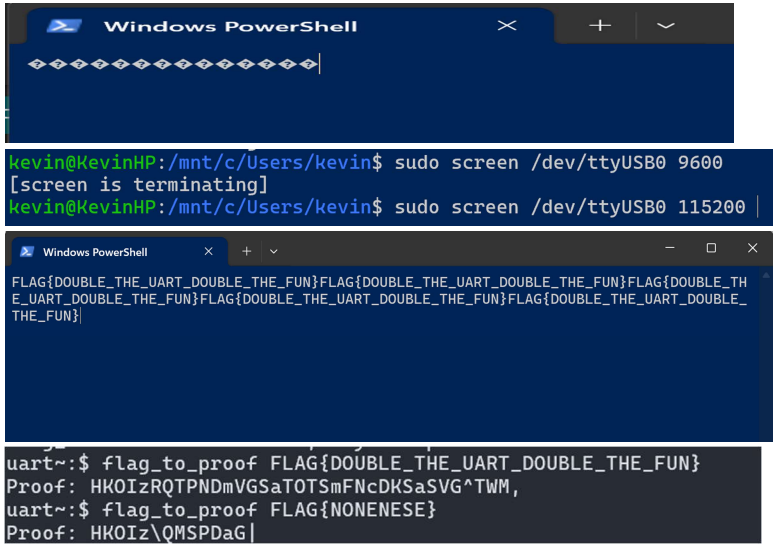
Method

1. **Hardware Setup**
    - Repeat the hardware setup for flag 1, using pins PB10, PB11



2. **Flag 5 and Proof**

- Connect to the UART debug shell using step 2 from flag 1 to reveal the data stream. Note that the baud rate of 9600 will not read the data. Make sure to switch to 115200
  FLAG{DOUBLE_THE_UART_DOUBLE_THE_FUN}
  Proof: HKOIzRQTPNDmVGSaTOTSmFNcDKSaSVG^TWM,



Remediation & Flags

Risk Scoring

To evaluate each finding, the following risk-scoring logic was applied:

- Impact: evaluates the resulting damage to ThanosTech's business priorities. This may include loss of intellectual property, potential loss of revenue, compromised firmware or sensitive data.
- Exploitability: assesses the effort required for an attacker to breach the device. This accounts for cost to the attacker, physical access, existing protection.
- Likelihood: considers the chances of real-world exploitation of the findings. This considers IoT trends, similar use cases etc.

Each of these criteria is scored on a scale of 1 to 10 and the final risk score is categorized as low, medium, or high.

Remediation & Flags Summary

Finding	Impact	Risk Score	Mitigation
Exposed UART Debug Interface: Finding 1&5	The exposed UART peripherals reveal development details and a sensitive data stream. This could reveal intellectual property and configurations.	High: UART is one of the most well-known attack vectors in IoT [1]. An unprotected UART pin is easily accessible, a $10 is enough to breach the debug shell.	If possible, the UART interface should be disable in production. Otherwise, consider removing UART headers. Another option is to enable secure bootloader to enforce authentication and block unathorized code execution. [2]
Weak Password Policy and Storage: finding 2	Depending on the purpose of the password, an attack might leak sensitive information, admin credentials, or proprietary data.	High: The UART data stream exposes a password hash without authentication. The hash is unsalted and easily reversible. Moreover, the original password easily guessable, making it more	Consider adopting a complex password policy, using a salted hashing algorithm like bcrypt and removing sensitive credentials from publicly available interfaces [3].

		susceptible to brute-force attacks. [3]	
Unprotected Flash Memory: findings 3 & 4	The internal and external flash memory might expose firmware leading to intellectual property and revenue loss.	SWD and unencrypted external flash chips are the most basic attacks for IoT devices. Exploit tools are as cheap as $20 and easy to setup. Moreover, the STM32 suite does not support hardware encryption, making it more vulnerable. [4]	If SWD is needed for operation, consider enabling readout protection at level 1 to prevent access to memory. [4]

Flags and Proofs – PCB 26

Finding	Proof	Flag
1	HKOIzW)LmTW5KMUaVVCSm[Nc)QSaSf)H\I|	FLAG{I'M_RX'ING_WHAT_YOU'RE_TX'ING}
2	HKOIzQCMm[NcaB`CBYaLS	FLAG{CAN_YOU_CRACK_ME}
3	HKOIzQQMaKCST^[G^RGAcIFSF|	FLAG{CONSIDER_ME_DEBUGGED}

4	HKOIzaYDSV^aYDSV^[GL]THSU|	FLAG{SWEET_SWEET_MEMORIES}
5	HKOIzRQTPNDmVGSaTOTSmFNcDKSaSVG^TWM,	FLAG{DOUBLE_THE_UART_DOUBLE_THE_FUN}

Conclusion

This assessment shows that the Node is critically vulnerable to easily actionable attacks. All these findings can be performed at a very low cost, with no time and no physical restriction. As these findings present a major risk to intellectual property and revenue, ThanosTech should strongly consider recalling all affected products as soon as possible. Further releases must enforce protect sensitive interfaces, authentication to configuration peripherals, enable readout protection and strong password policies.

References

1. Praetorian. (2021, June 29). *Why are JTAG and UART still effective attack vectors for IoT devices?* Praetorian. https://www.praetorian.com/blog/why-are-jtag-and-uart-still-effective-attack-vectors-for-iot-devices/
2. Particle. (n.d.). *IoT secure boot*. Particle. https://www.particle.io/iot-guides-and-resources/iot-secure-boot/
3. National Institute of Standards and Technology. (2024). *CVE-2024-3263*. NVD. https://nvd.nist.gov/vuln/detail/CVE-2024-3263
4. STMicroelectronics. (2020). *AN5156: Introduction to security for STM32 MCUs*. https://www.st.com/resource/en/application_note/an5156-introduction-to-security-for-stm32-mcus-stmicroelectronics.pdf